MIKROSERVIS MIMARISI

Outbox Pattern

Veritabanı ile Kafka arasında güvenilir event yayınlama

Örnek: bir e-ticaret sitesinde sipariş oluşturma

Spring Boot 3.5 · Apache Kafka · Spring Data JPA

GÜNDEM

Bu sunumda

01 Problem: dual write ve neden tehlikeli olduğu

02 Çözüm: outbox tablosu ve tek local transaction

03 Message relay: Polling Publisher ve CDC (Debezium)

04 Kafka ile outbox: partition, key, producer/consumer ayarları

05 Teslimat garantileri, idempotent consumer ve inbox

06 Sıralama, temizlik, izleme ve production pratikleri

07 Ne zaman kullanılır, alternatifler ve özet

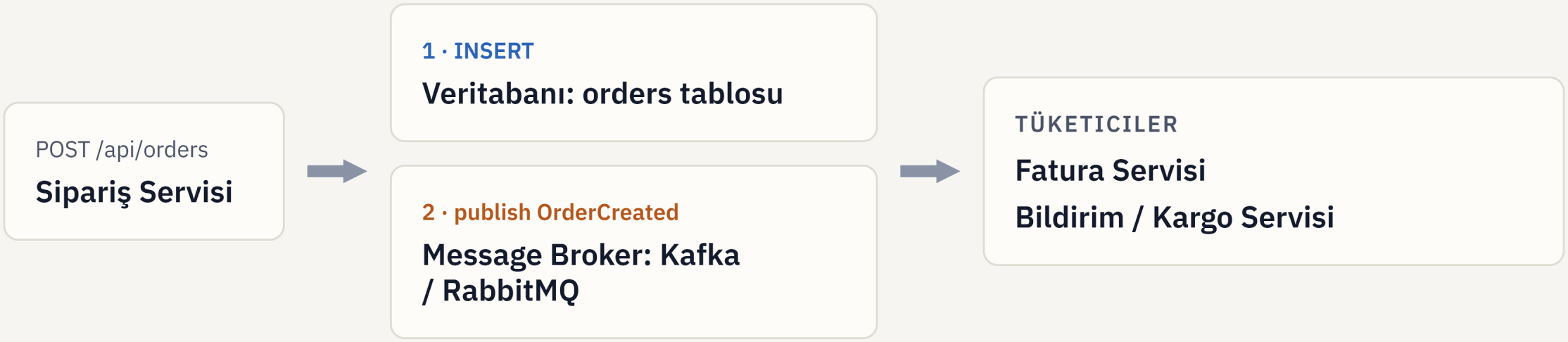
01

Problem: Dual Write

İki ayrı sisteme, ortak bir transaction olmadan yazmak

BAĞLAM

Sipariş oluşturulduğunda ne olmalı?



Servis siparişi kendi veritabanına yazar, diğer servisleri haberdar etmek için Kafka'ya bir event gönderir. Bu iki işlem birlikte başarılı olmalı: **ya ikisi de olur, ya hiçbiri.**

PROBLEM

Naif implementasyon

```
@Transactional
public OrderResponse createOrder(CreateOrderRequest req) {
    Order order = Order.create(req.customerId(), "TRY");
    orderRepository.save(order);  // 1) DB

    OrderCreatedEvent event = OrderCreatedEvent.from(order);
    publisher.publish("order-events", order.getId(),
        toJson(event), headers);  // 2) Kafka

    if (failAfterPublish) throw ...;  // DB rollback
    return OrderResponse.from(order);
}  // 3) commit burada gerçekleşir
```

@Transactional yalnızca DB'yi kapsar

Kafka bu transaction'ın parçası değildir. Gönderilen mesaj geri alınamaz.

Atomiklik yok

DB ve broker iki ayrı kaynak. Hangi sırayla yazılırsa yazılsın tutarsızlığa açık.

PROBLEM

Neler ters gidebilir?

KAYIP EVENT

DB commit, publish yok

Commit sonrası mesajı göndermeden önce pod çöker ya da broker erişilemez. Sipariş var, ama Fatura Servisi bundan hiç haberdar olmaz.

HAYALET EVENT

Publish var, DB rollback

Mesaj gönderildikten sonra commit başarısız olur (constraint ihlali, deadlock, timeout). Tüketiciler var olmayan bir sipariş için işlem yapar.

BELIRSIZLIK

Broker timeout

Send çağrısı zaman aşımına uğrar. Mesaj gitti mi gitmedi mi bilinmez. Retry yaparsan duplicate, yapmazsan kayıp riski vardır.

Bu hatalar nadirdir; ama e-ticarette tek bir kayıp sipariş event'i bile faturası kesilmemiş, kargoya verilmemiş bir sipariş ve **ciddi mutabakat sorunları** demektir.

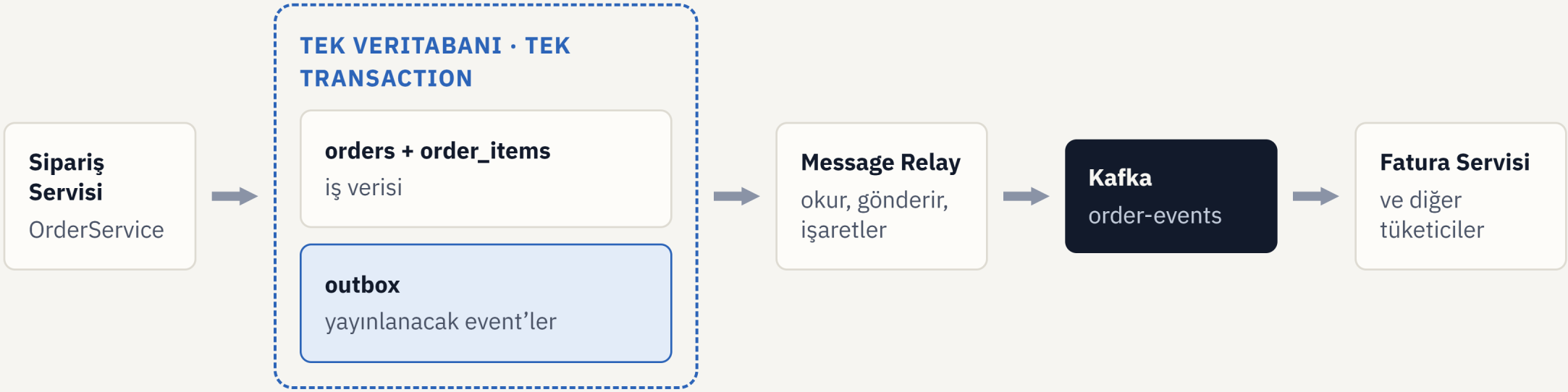
02

Çözüm: Outbox Tablosu

Event’i önce kendi veritabanına yaz, sonra yayınla

MIMARI

Pattern’in genel görünümü



İş verisi ve event **aynı commit ile** yazılır. Relay, outbox’taki kayıtları Kafka’ya taşır ve başarılı olanları PROCESSED olarak işaretler.

VERI MODELI · SCHEMA.SQL

Outbox tablosu

```
CREATE TABLE outbox (
  id             VARCHAR(36) PRIMARY KEY,
  aggregate_type VARCHAR(100) NOT NULL,
  aggregate_id   VARCHAR(100) NOT NULL,
  event_type     VARCHAR(100) NOT NULL,
  payload        CLOB NOT NULL,
  status         VARCHAR(20) NOT NULL,
  created_at     TIMESTAMP NOT NULL,
  processed_at   TIMESTAMP,
  retry_count    INT DEFAULT 0 NOT NULL,
  last_error     VARCHAR(1000)
);
CREATE INDEX ix_outbox_pending
  ON outbox (status, created_at);
```

Kolon	Amaç
id	Event id: tüketicide tekrarları ayıklama anahtarı
aggregate_type	Topic seçimi: Order → order-events
aggregate_id	Kafka key (orderId): sıra garantisi
event_type	OrderCreated, OrderCancelled…
payload	Event'in yazıldığı andaki JSON hali
status	Relay için yayın durumu: PENDING → PROCESSED

UYGULAMA · SPRING BOOT

Servis katmanı

```
@Service
public class OrderService {

    @Transactional
    public OrderResponse createOrder(CreateOrderRequest req) {
        Order order = Order.create(req.customerId(), "TRY");
        orderRepository.save(order);  // iş verisi

        OrderCreatedEvent e = OrderCreatedEvent.from(order);
        outboxWriter.write("Order", order.getId(),
                e.eventType(), e.eventId(), e);  // outbox

        return OrderResponse.from(order);  // broker'a hiçbir
şey gitmez
    }
}
```

- İki save aynı transaction içinde: ya ikisi de commit edilir ya hiçbiri
- Broker erişilemez olsa bile istek başarıyla tamamlanır
- İş mantığı Kafka'dan habersizdir, test etmesi kolaylaşır

İpucu

Event yazımını Spring'in ApplicationEventPublisher + @TransactionalEventListener (BEFORE_COMMIT) ile soyutlayabilirsiniz.

03

Message Relay

Outbox’tan broker’a: Polling Publisher ve Transaction Log Tailing

RELAY · YÖNTEM 1

Polling Publisher

```
@Scheduled(fixedDelayString = "${outbox.relay.fixed-delay}")
@SchedulerLock(name = "outboxRelay")  // ShedLock
public void relay() {
    transactionTemplate.executeWithoutResult(s -> {
        var batch = outboxRepository
                .findPendingForUpdateSkipLocked(100);

        for (OutboxEvent e : batch) {
            publisher.publish(topicOf(e), e.getAggregateId(),
                e.getPayload(), headers(e));  // ack bekle
            e.markProcessed();  // PROCESSED
        }
    });
}
```

ARTILAR

- Ek altyapı gerektirmez
- Her DB ile çalışır, anlaması kolay

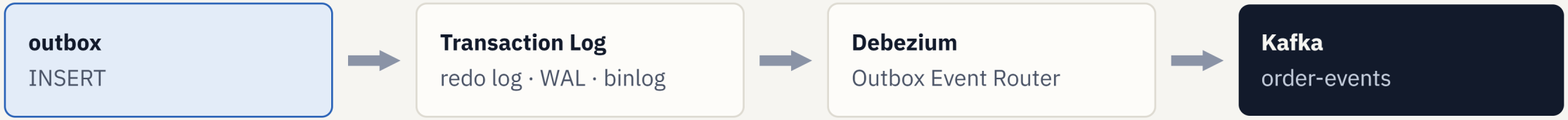
EKSILER

- DB'ye sürekli sorgu yükü
- Polling aralığı kadar gecikme
- Çoklu instance için kilit gerekir

```
SELECT * FROM outbox
WHERE status = 'PENDING'
ORDER BY created_at
FETCH FIRST 100 ROWS ONLY
FOR UPDATE SKIP LOCKED
```

RELAY · YÖNTEM 2

Transaction Log Tailing (CDC)



Debezium, veritabanının transaction log'unu bir replica gibi okur. Outbox'a eklenen her satırı yakalar ve aggregate_type'a göre ilgili topic'e yönlendirir.

ARTILAR

- Çok düşük gecikme
- Polling yükü yok
- Commit sırası korunur

EKSILER

- Kafka Connect işletimi
- DB log yetkileri
- Operasyonel karmaşıklık

Oracle için LogMiner/XStream, PostgreSQL için logical decoding, MySQL için binlog kullanılır.

KARŞILAŞTIRMA

Polling mi, CDC mi?

Kriter	Polling Publisher	CDC (Debezium)
Gecikme	Polling aralığına bağlı (yüzlerce ms – saniyeler)	Düşük, neredeyse gerçek zamanlı
DB yükü	Periyodik SELECT + UPDATE	Log okuma, sorgu yükü yok
Altyapı	Uygulama içinde, ekstra bileşen yok	Kafka Connect + Debezium cluster
Ölçeklenme	SKIP LOCKED / ShedLock ile sınırlı paralellik	Yüksek throughput için uygun
Outbox temizliği	PROCESSED ile işaretle, sonra sil	INSERT sonrası hemen silinebilir
Uygun olduğu durum	Orta hacim, hızlı başlangıç, sınırlı ops ekibi	Yüksek hacim, düşük gecikme, CDC altyapısı mevcut

Pratik öneri: Polling ile başlayın; hacim ve gecikme gereksinimleri arttığında outbox tablosunu değiştirmeden CDC'ye geçebilirsiniz.

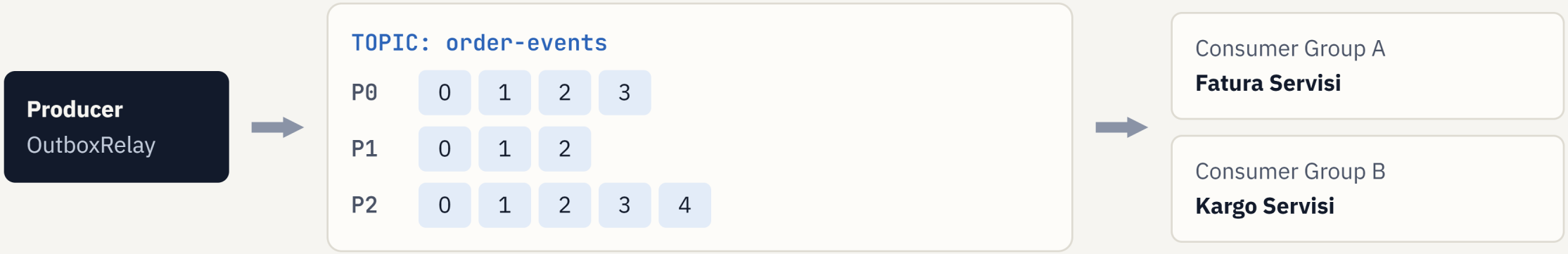
04

Kafka ile Outbox

Pattern’in en sık kullanıldığı broker ve bilinmesi gereken temel kavramlar

KAFKA · TEMELLER

Kafka’yı 1 dakikada hatırlayalım



Topic

Event’lerin yazıldığı mantıksal kanal. Kalıcıdır; okunan mesaj silinmez.

Partition

Topic’in paralel parçası. Sıralama yalnızca partition içinde garanti edilir.

Offset

Mesajın partition içindeki sıra numarası. Consumer nerede kaldığını offset ile takip eder.

Consumer Group

Her grup tüm mesajları alır; grup içinde partition’lar instance’lara bölünür.

KAFKA · OUTBOX ILIŞKISI

Kafka outbox’ta neyi sağlar, neyi sağlamaz?

MESSAGE KEY = AGGREGATE_ID

Aynı key aynı partition’a gider. OrderCreated → OrderPaid → OrderShipped sırası tüketicide korunur.

KALICILIK VE REPLAY

Mesajlar retention süresince saklanır. Yeni bir servis eski event’leri baştan okuyabilir; hatalı consumer offset geri alınarak yeniden iletilebilir.

KAFKA TRANSACTION ≠ DB TRANSACTION

Kafka’nın transaction ve exactly-once özelliği yalnızca Kafka içindeki yazmaları kapsar (topic’ler ve offset’ler). Veritabanındaki bir INSERT ile atomik hale getirilemez.

Bu yüzden Kafka tek başına dual write problemini çözmez; köprüyü outbox kurar. Pattern RabbitMQ, SQS gibi broker’larla da aynı şekilde çalışır; ancak partition bazlı sıralama ve replay yetenekleri nedeniyle en yaygın eşleşme Kafka’dır.

KAFKA · SPRING BOOT AYARLARI

Producer ve consumer konfigürasyonu

```
# application.yml
spring.kafka:
  producer:  # Outbox Relay
    acks: all  # tüm ISR'ler onaylasın
    retries: 2147483647
    properties:
      "[enable.idempotence]": true  # retry duplicate'i yok
      "[max.in.flight.requests.per.connection]": 5
  consumer:  # Fatura Servisi
    enable-auto-commit: false
    auto-offset-reset: earliest
    isolation-level: read_committed
  listener:
    ack-mode: record
```

acks=all + idempotence: Relay'in retry'ları Kafka'da duplicate oluşturmaz, yazılan mesaj kaybolmaz.

Producer idempotence yetmez: Relay çöküp yeniden başlarsa aynı kayıt yeni bir send olarak gider. Bunu consumer'daki inbox yakalar.

Auto commit kapalı: Offset, DB işlemi commit edildikten sonra ilerletilir; işlenmeden kaybolan mesaj olmaz.

Hata yönetimi: DefaultErrorHandler + Dead Letter Topic ile zehirli mesajlar order-events.DLT'ye alınır.

05

Teslimat Garantileri

Kayıp yok, ama duplicate olabilir

GARANTILER

Outbox = at-least-once teslimat

Relay mesajı Kafka'ya gönderir, ama kaydı PROCESSED yapamadan çökerse ne olur?

1. Outbox kaydı okunur
2. Kafka'ya gönderilir, ack alınır
3. Relay çöker, PROCESSED yazılamaz
4. Yeniden başlar, aynı event tekrar gider

SONUÇ

Exactly-once sağlanmaz

Outbox, event'in en az bir kez yayınlanacağını garanti eder. Kayıp riski ortadan kalkar, fakat duplicate riski kabul edilir.

Bu yüzden pattern'in diğer yarısı tüketici tarafındadır: consumer'lar aynı mesajı birden fazla kez alsa bile doğru davranmalıdır.

TÜKETICI TARAFI

Idempotent consumer ve inbox tablosu

```
@KafkaListener(topics = "order-events",
               groupId = "invoice-service")
@Transactional
public void on(ConsumerRecord<String, String> rec) {
    String eventId = header(rec, "eventId");

    if (inboxRepository.existsById(eventId)) {
        return;  // zaten işlendi
    }

    invoiceRepository.save(Invoice.forOrder(parse(rec)));
    inboxRepository.save(new InboxMessage(eventId, type));
}  // iş + inbox aynı transaction'da
```

- Her event’in benzersiz bir id’si olmalı (outbox.id)
- İşlenen id’ler inbox tablosunda tutulur; PK çakışması duplicate’i engeller
- Fatura ve inbox kaydı aynı local transaction’da yazılır
- Alternatif: doğal olarak idempotent işlemler (upsert, “durumu X yap”)

Outbox üreticiyi, inbox tüketiciyi güvence altına alır.

06

Production Pratikleri

Sıralama, temizlik, hata yönetimi ve izleme

PRODUCTION

Dikkat edilmesi gerekenler

SIRALAMA

Kafka key olarak aggregate_id (orderId) kullanın; aynı siparişin event'leri aynı partition'a düşer ve sırası korunur. Relay'de created_at sırasıyla okuyun.

TEMIZLIK

Outbox sürekli büyür. İşlenmiş kayıtları periyodik bir job ile silin veya arşivleyin. Yüksek hacimde tarih bazlı partition kullanıp eski partition'ları drop etmek daha verimlidir.

ZEHIRLI MESAJLAR

Sürekli başarısız olan event tüm kuyruğu kilitlememeli. retry_count eşiği aşılınca kaydı FAILED durumuna alın ve alarm üretin.

İZLEME

Bekleyen kayıt sayısı ve en eski bekleyen kaydın yaşı en önemli metriklerdir. Yaş artıyorsa relay takılmış demektir.

PRODUCTION

Payload tasarımı

Snapshot yazın

Payload’ı event anındaki haliyle saklayın. Relay çalışırken entity’yi yeniden okumak, arada değişmiş veriyi yayınlamanıza yol açar.

Versiyonlayın

Şema evrimi için schemaVersion alanı ekleyin veya Avro + Schema Registry kullanın.

Domain event, entity değil

Tablonun ham hali yerine anlamlı bir event yayınlayın; iç şemanız dışarıya sızmasın.

Hassas veri

Kart bilgisi, TCKN, açık adres gibi alanları payload’a gereksiz yere koymayın.

```
{
  "eventId": "9f1c2e7a-...",
  "eventType": "OrderCreated",
  "schemaVersion": 1,
  "occurredAt": "2026-10-17T08:21:04Z",
  "orderId": "dec71fbd-...",
  "customerId": "C-55120",
  "totalAmount": 2999.80,
  "currency": "TRY",
  "items": [
    { "productId": "P-100", "quantity": 2,
      "unitPrice": 1499.90 }
  ]
}
```

UÇTAN UCA

Tam akış

#	Kimden → kime	Adım
1	Sipariş Servisi → DB	INSERT orders + outbox, COMMIT
2	Relay → DB	Bekleyenleri oku (SKIP LOCKED)
3	Relay → Kafka	publish (key = orderId)
4	Kafka → Relay	ack
5	Relay → DB	status = PROCESSED
6	Kafka → Fatura Servisi	deliver
7	Fatura Servisi	inbox kontrol + fatura oluştur

07

Değerlendirme

Ne zaman kullanılır, alternatifler ve özet

KARAR

Ne zaman kullanmalı?

KULLANIN

- Bir state değişikliği ile bir event'in birlikte tutarlı olması gerekiyorsa
- Saga / choreography tabanlı iş akışlarında
- Event kaybının mutabakat sorunu yaratacağı domain'lerde
- Broker'ın geçici olarak erişilemez olabileceği ortamlarda

GEREKMEYEBILIR

- Event kaybının tolere edildiği durumlarda (log, analitik, metrik)
- Servisin kendi veritabanı yoksa veya yazma yapmıyorsa
- Zaten event sourcing kullanılıyorsa (event store doğal outbox'tır)
- Senkron, tek servisli basit CRUD uygulamalarında

ALTERNATIFLER

Benzer problemi çözen yaklaşımlar

Yaklaşım	Fikir	Dezavantaj
2PC / XA	DB ve broker'ı tek dağıtık transaction'a bağlamak	Yavaş, kırılgan, Kafka desteklemez
Listen to Yourself	Önce event'i yayınla, servis kendi event'ini tüketip DB'ye yazsın	Read-your-writes kaybolur, karmaşık akış
Event Sourcing	Durumun kendisi event'lerden oluşur, event store = kaynak	Büyük paradigma değişimi
Doğrudan CDC	İş tablolarının değişikliklerini doğrudan yayınlamak	İç şema dışarı sızar, domain anlamı kaybolur
Transactional Outbox	Event'i iş verisiyle aynı transaction'da kaydet, ayrıca yayınla	Ek tablo, relay ve idempotent consumer gerekir

ÖZET

Akılda kalması gerekenler

1 · Problem

DB'ye yazmak ve broker'a göndermek iki ayrı işlemdir; dual write kayıp ya da hayalet event üretir.

2 · Çözüm

Event'i iş verisiyle aynı local transaction'da outbox tablosuna yaz.

3 · Relay

Polling ile basit başla; hacim artınca Debezium/CDC'ye geç.

4 · Garanti

At-least-once: kayıp yok, duplicate mümkün.

5 · Tüketici

Consumer idempotent olmalı; inbox tablosu ile duplicate'leri ele.

6 · İşletim

Sıralama için aggregate key, düzenli temizlik, bekleyen kayıt yaşını izle.

Teşekkürler

Sorular?

KAYNAKLAR

Chris Richardson — Microservices Patterns (Manning) ve microservices.io
Debezium Dokümantasyonu — Outbox Event Router
Martin Kleppmann — Designing Data-Intensive Applications (O’Reilly)